

RO47005 Planning and Decision Making Project

Tommaso Calzolari 6430600
Leonardo Pedretti 6432891
Simone Pigliapoco 6403344
Kevin Stehouwer 6304915

I. INTRODUCTION

A complete motion planning pipeline for a quadrotor has been developed to enable autonomous navigation in a maze-like environment containing multiple obstacles. The environment includes obstacles with known locations, such as the walls of the maze, as well as obstacles with initially unknown locations that must be detected and avoided during flight. Global path planning is performed using a sampling-based Rapidly-exploring Random Tree (RRT*) algorithm, which computes a collision-free reference path between a predefined start location A and goal location B . This global plan is subsequently followed by a local Model Predictive Control (MPC) scheme, which accounts for the quadrotor dynamics and actuator constraints while enabling online adaptation in the presence of newly observed obstacles. The quadrotor dynamic model employed in the MPC formulation is based on established quadrotor dynamics described in [1], while the simulation environment and physics are provided by the `gym-pybullet-drones` framework [9]. The task of the quadrotor is to autonomously move from location A to location B while avoiding collisions by computing and tracking a feasible, collision-free trajectory.

The choice of combining RRT* for global planning with MPC for local planning is motivated by their complementary properties. Sampling-based planners such as RRT* operate directly in continuous state spaces and are particularly effective in cluttered and high-dimensional environments [4]. This makes it well suited for computing collision-free global reference paths in maze-like environments without requiring discretization, as would be the case for graph-search methods such as Dijkstra or A*. However, RRT* does not explicitly consider system dynamics or actuator constraints. To address this limitation, Model Predictive Control (MPC) is employed as a local planner and controller. MPC explicitly incorporates a model of the quadrotor dynamics and enforces state and input constraints while repeatedly optimizing control actions over a finite prediction horizon [10]. This enables accurate and fast trajectory tracking and online adaptation in the presence of newly observed obstacles, making MPC a suitable choice for local planning and control. Compared to sampling-based control methods such as Model Predictive Path Integral (MPPI) control, MPC provides a more direct and systematic way to handle hard state and input constraints, which is particularly well suited for the considered environment with static obstacles and a continuous control objective. From

a state-of-the-art perspective, RRT* and MPC are widely used baselines, but many current deployments build on improved variants of both. On the planning side, recent work accelerates convergence and/or supports frequent replanning through informed and incremental sampling strategies, such as Informed RRT* [2], as well as real-time replanning extensions like RRTX [8]. On the control side, quadrotor implementations increasingly adopt nonlinear and robust MPC formulations to better handle modeling errors and disturbances while preserving constraint satisfaction, with tube-based MPC being a representative approach demonstrated in real-time settings [3].

II. PROBLEM DEFINITION

The objective is to autonomously navigate a quadrotor from a start location A to a goal location B in a three-dimensional environment containing static obstacles, by computing and tracking a collision-free trajectory. The workspace and configuration space will be defined in which the robot operates. The workspace of the drone is defined as:

$$\mathcal{W} = \mathbb{R}^3$$

The configuration space of the drone is defined as:

$$\mathcal{C} = \mathbb{R}^3 \times SO(3)$$

$$\mathcal{C} = SE(3)$$

- $\mathbb{R}^3$: the set of all possible positions of the drone, this is the 3D Workspace.
- $SO(3)$: the Special Orthogonal Group representing all possible orientations of the drone.
- $SE(3)$: the Special Euclidean Group representing all possible positions and orientations of the drone

Overall the robot has 3 translational degrees of freedom and 3 rotational degrees of freedom so the configuration space is 6-dimensional.

We model the quadrotor (X configuration) with the state vector([11]):

$$\mathbf{x} \triangleq [\mathbf{p}^\top \quad \mathbf{v}^\top \quad \boldsymbol{\eta}^\top \quad \boldsymbol{\omega}^\top]^\top$$

where $\mathbf{p} = [x, y, z]^\top$ and $\mathbf{v} = [v_x, v_y, v_z]^\top$ denote the position and linear velocity in the inertial frame, $\boldsymbol{\eta} = [\phi, \theta, \psi]^\top$ are the Euler angles, and $\boldsymbol{\omega} = [p, q, r]^\top$ is the body-frame angular velocity.

Each rotor i produces thrust and drag torque $T_i = C_T \omega_i^2$ and $Q_i = C_D \omega_i^2$, yielding total thrust $T = C_T \sum_{i=1}^4 \omega_i^2$ and body moments $\mathbf{M}^b = [M_x, M_y, M_z]^\top$ (given in the mixer equations below). The dynamics are

$$m\ddot{\mathbf{p}} = R\mathbf{F}^b - mg\mathbf{e}_3, \quad I\dot{\boldsymbol{\omega}} = \mathbf{M}^b - \boldsymbol{\omega} \times (I\boldsymbol{\omega}),$$

with standard Euler-angle kinematics relating $\boldsymbol{\omega}$ and $\dot{\boldsymbol{\eta}}$. Linearizing around hover ($\phi, \theta \approx 0, T \approx mg$) gives $\ddot{x} \approx g\theta$, $\ddot{y} \approx -g\phi$, and $\ddot{z} \approx \frac{1}{m}\Delta T$, where $\Delta T = T - mg$ is the thrust deviation from hover.

Accordingly, we use the continuous-time linear model $\dot{\mathbf{x}} = A_c \mathbf{x} + B_c \mathbf{u}$ with

$$\mathbf{u} = [T \quad \tau_\phi \quad \tau_\theta \quad \tau_\psi]^\top,$$

and the matrices A_c and B_c reported in (1).

$$A_c = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & g & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -g & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B_c = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ \frac{1}{m} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & \frac{1}{I_{xx}} & 0 & 0 \\ 0 & 0 & \frac{1}{I_{yy}} & 0 \\ 0 & 0 & 0 & \frac{1}{I_{zz}} \end{bmatrix}.$$

Using a forward Euler discretization with sampling time T_s , the continuous-time model

$$\dot{\mathbf{x}}(t) = A_c \mathbf{x}(t) + B_c \mathbf{u}(t)$$

yields the discrete-time dynamics

$$\mathbf{x}_{k+1} = \mathbf{x}_k + T_s (A_c \mathbf{x}_k + B_c \mathbf{u}_k) = \underbrace{(I + T_s A_c)}_{A_d} \mathbf{x}_k + \underbrace{(T_s B_c)}_{B_d} \mathbf{u}_k, \quad (2)$$

where I is the identity matrix and $\mathbf{x}_k \approx \mathbf{x}(kT_s)$, $\mathbf{u}_k \approx \mathbf{u}(kT_s)$. This discrete model will be used in the constraint of the MPC formulation as explained in section III.

The base of the simulation environment and the model are handled by the `gym-pybullet-drones` library, which is

used as the simulation backbone for the motion planning algorithm developed in this project.¹

III. MOTION PLANNING METHOD

For the problem stated in section II, the incremental sampling-based method Rapidly-exploring Random Tree (RRT*) will be used for global path planning, since it performs well in high-dimensional configuration spaces and provides asymptotic optimality and probabilistic completeness as the number of iterations approaches infinity, meaning that the probability of finding a feasible path approaches one if such a path exists, and that the cost of the solution converges to the optimal solution over time. However, for this project, a variant of RRT* has been used to speed up the path planning process by returning the first feasible path found. By doing this, as the number of iterations approaches infinity, the probability of finding a feasible path still approaches one if such a path exists but, the cost of the solution does not converge to the optimal solution over time since it does not update the path after finding the first feasible solution. In addition, Model Predictive Control (MPC) will be used for local path planning, as it enables to avoid encountered obstacles during flight. MPC plans over a finite prediction horizon, explicitly handles constraints, and recomputes the control inputs at each time step based on updated state information, meaning that it is perfectly suited for the objective.

A. Rapidly-exploring Random Tree*

The implemented RRT* method takes as input a start and goal position in Cartesian coordinates, a list of known obstacles, the sampling domain, the expansion distance, the goal sampling rate, and the maximum number of iterations. Using these inputs, it performs the following steps to construct a feasible path:

- 1) Sample a random state within the sampling domain and determine the nearest node already contained in the tree (the goal sampling rate determined how often this sample will be the goal position).
- 2) Generate a new node by steering from the nearest node toward the sampled state with a step size equal to the expansion distance. In addition add the cost (distance from start to new node) to the new node.
- 3) Check the new edge (and the new node) for collisions with the known obstacles. If a collision is detected, discard the new node and sample again.
- 4) Identify the set of neighboring nodes within a pre-defined radius of the new node, and select as parent the neighbor that yields the lowest cost-to-come while remaining collision-free.
- 5) Rewire the tree: for each neighbor, if connecting it through the new node reduces its cost-to-come and the connection is collision-free, update its parent accordingly.

¹<https://github.com/utiasDSL/gym-pybullet-drones>

- 6) If the new node lies within the expansion distance of the goal, check the edge from the new node to the goal for collisions. If collision-free, a feasible solution path has been found and will be returned.

If no feasible path is found within the specified maximum number of iterations, the RRT planner returns no solution and the planning loop is restarted until a feasible path is eventually found.

B. Model Predictive Control

Model Predictive Control (MPC) is used as the local planner. It takes the RRT* path as a reference and computes control inputs that track this trajectory while enforcing system and actuator constraints following the model described in section II. Unlike RRT*, which plans using only the known maze geometry, MPC explicitly incorporates additional obstacles that are introduced online (randomly placed along or near the nominal route) by embedding them as constraints in the optimization problem. A receding horizon of 30 steps is adopted to provide sufficient look-ahead in narrow passages and densely cluttered regions. Obstacle avoidance is handled through a separating-hyperplane formulation that yields a local linear approximation of the safety constraints. For each obstacle detected within a prescribed sensing radius, the controller computes a normal vector from the obstacle boundary toward the drone's current position. This normal defines a supporting half-space, expressed as the linear inequality $Ax \leq b$, which constrains the drone to remain on the safe side of a hyperplane tangent to the obstacle ([7]). The hyperplanes are recomputed at every control step using the drone's measured state, resulting in a time-varying convex feasible region that moves with the vehicle through the environment. To mitigate infeasibility in narrow passages, where the RRT* reference and the randomly generated obstacles may induce conflicting constraints, soft constraints are introduced via a slack variable. The Clarabel solver is adopted due to its numerical robustness and its efficiency in solving convex optimization problems, which contributes to smoother control inputs. Finally, a Terminal Set and Terminal Cost (inspired by [12]) are incorporated into the optimization to satisfy the Lyapunov stability criteria, effectively ensuring the stability of the system over an equivalent infinite horizon.

IV. RESULTS

As mentioned in Section I, *gym-pybullet-drones* has been used as simulation framework, since it provides a realistic quadrotor model in a full 3D physics engine. The system will be evaluated in a top-down S-shaped maze with tunnel and "stair" like obstacles inside, with random obstacles appearing along the RRT* reference path. Initially, the RRT* planner (implemented by Simone Pigliapoco 6403344, Kevin Stehouwer 6304915) and the MPC controller (implemented by Tommaso Calzolari 6430600, Leonardo Pedretti 6432891) were developed and tested in parallel. After achieving stable performance, both methods were integrated and tested in a maze without random obstacles. Introducing random obstacles increased the difficulty, requiring a terminal set and

terminal cost in the MPC and further tuning of the Q and R matrices. This setup became our baseline and achieved 100% success. We then moved to a harder maze with tunnels and "stair-box" obstacles. These were modeled in both RRT* and MPC, making the task significantly more challenging due to tight spaces and real-time obstacle avoidance. To improve feasibility, we switched to the Clarabel solver and replaced the fixed convex feasible region with a time-varying one.

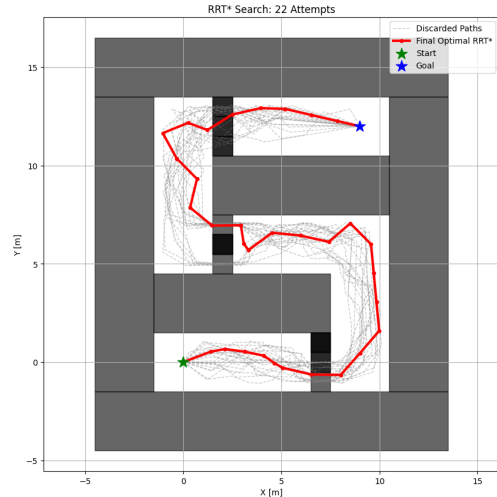


Fig. 1. RRT* reference trajectory

Figure 1 illustrates the path produced by the RRT* planner in the S-shaped maze. The start and goal positions are marked with a green and blue star, respectively, while the final selected RRT* path is shown in red. The dashed gray curves correspond to candidate solutions explored during the search and later discarded, either because they were longer or because they introduced overly sharp turns that are unsuitable for the subsequent spline fitting and for the quadrotor's dynamic constraints. Overall, the figure shows that the planner reliably finds a collision-free route through the maze, while the spline-based post-processing acts as a geometric filter that favors smoother, dynamically feasible trajectories. Although the planner is based on RRT* (see Section III), the implementation terminates as soon as a first collision-free solution reaching the goal is found and selected. As a result, the "anytime" property is exploited only until a satisfactory solution is obtained, and no further refinement (i.e., additional sampling and rewiring) is performed afterwards. This design choice has been adopted because, reaching a near-optimal RRT* solution requires extensive sampling and rewiring, which is very time-consuming in the current environment. Moreover, after a path is found, it is post-processed by a spline smoother that often rejects trajectories with sharp turns or tight passages. In that case the planner must restart and search for a new solution, making an anytime-optimal refinement impractical (often taking tens of minutes).

Therefore, the method stops once it obtains a first feasible

TABLE I

RRT* METRICS FOR THE FIRST FEASIBLE SOLUTION AND THE FINAL (COMPLETED) SOLUTION ACROSS ENVIRONMENTS.

Environment	RRT* first solution			RRT* complete		
	Time per run [s]	Path length [m]	Spline rejects [#]	Time per run [s]	Path length [m]	Spline rejects [#]
Baseline (Standard Maze)	≈ 0.031	≈ 37.8	1.4	≈ 189.7	≈ 38.7	3.2
Project (Tunnels & Stairs)	≈ 0.904	≈ 42.1	15.1	≈ 102.3	≈ 48.1	6.1

path that passes the spline post-processing. As evidenced in Table I, waiting for a fully optimized solution is computationally prohibitive (> 100 s) compared to the first feasible instance (≈ 0.9 s). Although the greedy ‘first solution’ incurs a higher rate of spline rejections due to sharper geometric turns, the extremely low search time renders these retries computationally negligible. By stopping early, a valid reference trajectory is secured within the real-time budget, ensuring the robustness of the downstream MPC controller without the diminishing returns of exhaustive optimization.

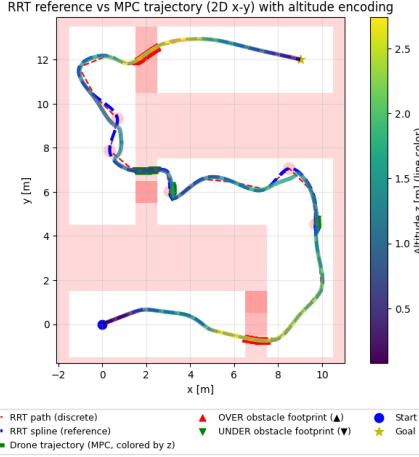


Fig. 2. RRT* reference trajectory (blue dashed) and actual drone trajectory with obstacle avoidance

In Fig. 2, the discrete RRT* waypoints are shown in red, while the spline-interpolated reference used by the MPC is shown in blue. The thicker, color-coded curve represents the executed drone trajectory, where the colormap encodes the altitude z . With an appropriate tuning of the Q and R weights, the controller follows the reference closely and deviates only when necessary to avoid obstacles that are not explicitly handled by the RRT* planner (highlighted by the markers along the reference). Even in narrow sections, the MPC produces smooth avoidance maneuvers. This behavior is enabled by the time-varying convex feasible set, which is recomputed online based on the current drone state and the positions of nearby obstacles.

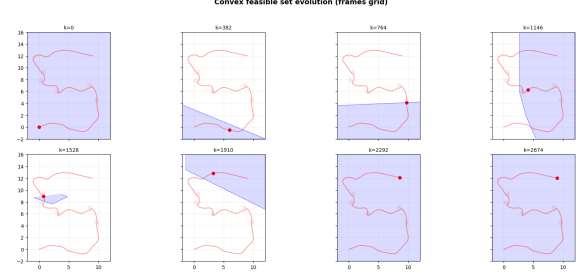


Fig. 3. Convex feasible region for different drone positions on the same run. Video available at <https://youtube.com/shorts/ie7j0NexTvI?feature=share>.

In Fig. 3, the evolution of the time-varying convex feasible set at several representative MPC steps along the flight have been illustrated. The shaded blue area denotes the admissible region in the x - y plane, while the dashed line(s) correspond to the supporting half-space constraints generated by nearby obstacles. As the drone approaches one or more obstacles (either from the fixed maze layout or from the additional randomly placed obstacles), the resulting half-spaces become more restrictive and the feasible region can shrink significantly, especially in tight passages. This reduced admissible set increases the difficulty of the optimization problem, leading to slower convergence and occasional non-optimal termination. To improve robustness, we therefore replaced OSQP with Clarabel; In Table II we observe that the first-order solver OSQP frequently hits the maximum iteration budget, indicating slow or unreliable convergence when the feasible set becomes tight. While first-order methods can be fast on well-conditioned problems, they typically trade accuracy for speed and are more sensitive to ill-conditioning introduced by restrictive obstacle constraints. As a consequence, OSQP does not consistently return an optimal status, especially in proximity to obstacles. The reported ‘‘88,5% optimal’’ rate should therefore be interpreted with care: non-optimal terminations are not uniformly distributed over the trajectory, but occur predominantly when the drone is close to obstacles and the convex feasible region shrinks. In these segments, the controller may fail to generate a reliable avoidance maneuver, resulting in a collision.

TABLE II
MPC SOLVER PERFORMANCE.

Solver	Avg. iter.	Avg. solve time (ms)	Optimal (%)	Non-opt. (%)
Clarabel	≈ 8.7	≈ 15.4	≈ 100.0	≈ 0.0
CVXPY / OSQP	<i>max.Iter</i>	≈ 6.5	≈ 88.5	≈ 11.5

Overall, after changing the solver to Clarabel, with this configuration we achieved approximately 92% success. This represents a slight decrease with respect to the baseline scenario (which does not include the tunnel and stairs inside the maze), while still providing reliable performance in the considered environment. In addition, a more challenging setting featuring narrower corridors and a higher number of random obstacles (more than eight) has been evaluated. In this case, the trade-off between environment complexity and controller performance becomes evident: the solver, combined with the time-varying convex feasible set, did not consistently return an optimal solution at every time step, requiring more solver iterations and longer solve times per step, and ultimately reducing the success rate to approximately 74%. The corresponding statistics are reported in Table III.

TABLE III
MPC PERFORMANCE ACROSS DIFFERENT ENVIRONMENT COMPLEXITIES
USING THE CLARABEL SOLVER.

Environment	Avg. iter.	Avg. solve time (ms)	Solver Optimal (%)	Task Success (%)
Baseline (Standard Maze)	≈ 6.2	≈ 9.1	≈ 100.0	≈ 100.0
Project (Tunnels & Stairs)	≈ 8.7	≈ 15.4	≈ 100.0	≈ 92.0
Complex (Narrow / > 8 Obs.)	≈ 11.4	≈ 18.9	≈ 96.5	≈ 74.0

Finally, here is a video of the final implementation.²

V. DISCUSSION AND CONCLUSION

In this project, a complete motion planning and control pipeline for a quadrotor operating in a cluttered three-dimensional environment has been developed and validated in simulation. The proposed architecture combines a global planner based on RRT* with a local planner and controller based on Model Predictive Control. This separation of roles proved effective: the global planner efficiently computed a collision-free reference path in a maze-like environment with known static obstacles, while the local MPC controller ensured dynamic feasibility, accurate trajectory tracking, and online adaptation to additional obstacles encountered during execution. Rather than investigating alternative planners or controllers, we were confident in the RRT*+MPC architecture and therefore concentrated our efforts on refining this pipeline, including testing different RRT* variants, tuning the MPC formulation (e.g., terminal cost and terminal set design), comparing solvers, and progressively increasing the complexity of the environment. Regarding the global planning method, several variants of RRT and RRT* were evaluated in order to identify the most suitable solution for the given application. The final selection was made between the standard, academically established RRT* algorithm and a simplified variant of the same approach. In particular, the simplified version terminates as soon as a feasible path to the goal is found and returns the most recent trajectory, which is locally optimal but not necessarily globally optimal. Despite its reduced optimality guarantees and early-stopping behavior, the simplified RRT* variant was ultimately chosen,

as it better suits the requirements of our application for the following reasons:

- The globally optimal trajectories produced by the standard RRT* algorithm were often extremely close to obstacles in the maze-like environment. This resulted in trajectories that occasionally rendered the subsequent MPC-based obstacle avoidance infeasible or numerically unstable.
- The simplified RRT* variant required significantly lower computational effort and planning time, making it more suitable for real-time or near real-time execution.

Therefore, considering both the geometry of the environment and the downstream constraints imposed by the MPC controller, the simplified version of RRT* was selected as the global planner.

On the control side, MPC successfully tracked this reference using a quadrotor model linearized around hovering, explicitly enforcing state and input constraints. Importantly, the MPC formulation was able to incorporate new, randomly generated obstacles that were not visible to the global planner. When such obstacles were encountered, the controller locally modified the trajectory through convex constraints, avoided collisions in a computationally efficient manner, and smoothly returned to the nominal global path once feasible. The results shown in figure 2 and in table III shown the reliability of this hierarchical approach reaching very high success rate in complex environment with random obstacles generation. When the environment becomes tighter, however, the success rate decreases (Table III), suggesting that feasibility and solver robustness become limiting factors under reduced clearance and frequent constraint activation. Future work could therefore focus on improving both the global plan quality and the local closed-loop robustness. On the planning side, faster-converging and replanning-oriented variants such as Informed RRT* and BIT* can improve solution quality under tight corridors, while incremental replanning methods such as RRTX can better support frequent updates when the map changes online [2]. On the control side, replacing the linear MPC with nonlinear and/or robust formulations (e.g., tube MPC) can increase resilience to modeling errors and disturbances in narrow passages [3]. Additionally, moving obstacles could be added, requiring the MPC to account for obstacle motion and possibly extending the global planner to handle time-dependent constraints. Furthermore, additional random obstacles could be introduced that are exclusively known to the MPC, further emphasizing the role of local online adaptation. Finally, the framework could be extended to multi-drone scenarios, where multiple quadrotors must generate distinct trajectories, coordinate their motion, and avoid collisions with one another. Such extensions would move the system closer to realistic multi-agent autonomous navigation problems and provide a richer testbed for advanced planning and control strategies. [6][5]

REFERENCES

- [1] Matthias Faessler, Antonio Franchi, and Davide Scaramuzza. “Differential flatness of quadrotor dynamics

²<https://www.youtube.com/watch?v=0f1BAWo5be4>

- subject to rotor drag for accurate tracking of high-speed trajectories”. In: *IEEE Robotics and Automation Letters* (2016).
- [2] Jonathan D. Gammell, Siddhartha S. Srinivasa, and Timothy D. Barfoot. “Informed RRT*: Optimal Sampling-Based Path Planning Focused via Direct Sampling of an Admissible Ellipsoidal Heuristic”. In: *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2014, pp. 2997–3004. DOI: 10.1109/IROS.2014.6942976.
 - [3] Haimin Hu et al. “Real-Time Tube MPC Applied to a 10-State Quadrotor Model”. In: *2018 Annual American Control Conference (ACC)*. 2018, pp. 3135–3140.
 - [4] Sertac Karaman and Emilio Frazzoli. “Sampling-based algorithms for optimal motion planning”. In: *The International Journal of Robotics Research* 30.7 (2011), pp. 846–894.
 - [5] Mykel J. Kochenderfer, Tim A. Wheeler, and Kyle H. Wray. *Algorithms for Decision Making*. Cambridge, MA: MIT Press, 2022. ISBN: 9780262047013.
 - [6] Steven M. LaValle. *Planning Algorithms*. Cambridge, UK: Cambridge University Press, 2006. ISBN: 9780521862059.
 - [7] Sikang Liu et al. “Planning Dynamically Feasible Trajectories for Quadrotors Using Safe Flight Corridors in 3-D Complex Environments”. In: *IEEE Robotics and Automation Letters* 2.3 (2017), pp. 1688–1695. DOI: 10.1109/LRA.2017.2663526.
 - [8] Michael W. Otte and Emilio Frazzoli. “RRT^X: Asymptotically Optimal Single-Query Sampling-Based Motion Planning with Quick Replanning”. In: *The International Journal of Robotics Research* 35.7 (2016), pp. 797–822. DOI: 10.1177/0278364915594679.
 - [9] Jacopo Panerati et al. “Learning to Fly—a Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-agent Quadcopter Control”. In: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* 1.1 (Jan. 2021), pp. 1–8. DOI: 10.0000/000000.
 - [10] James B. Rawlings, David Q. Mayne, and Moritz Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2017.
 - [11] Francesco Sabatino. “Quadrotor control: modeling, nonlinear control design, and simulation”. Master’s Degree Project. Stockholm, Sweden: KTH Royal Institute of Technology, June 2015. URL: https://www.kth.se/polopoly_fs/1.588039.1600688317!/Thesis%20KTH%20-%20Francesco%20Sabatino.pdf (visited on 01/12/2026).
 - [12] Moji Shi and Benjamin Piet. *A model predictive control approach for Quadrotor collision avoidance*.